# A playbook for a first Rust component in a C++ control plane

Each step carries the measurement that justifies it, and the band below carries what the migration does not buy

### 1 Choose by interface

Not by size or by how hot it is. What decides the cost is how narrow and scalar the API already is.

**MEASURED**
11 entry points and 231 lines, already scalar.

### 2 Design the C surface

An opaque handle, scalar-typed operations, and a status convention. This is the step needing real design decisions.

**WHERE THE EFFORT IS**
Rich objects, templates, callbacks: design out.

### 3 Isolate the ABI

Every raw-pointer operation goes in a separate seam crate depending on an unmodified core, under one documented contract.

**MEASURED**
0 unsafe in the core, 9 functions in the seam.

### 4 Measure twice

The round-trip, which says what a deployment feels, and the component, which says what the language does. Neither substitutes.

**WHY**
A round-trip is mostly the OS, not the language.

## What this buys

The migrated component stops contributing new use-after-free, double-free, iterator-invalidation, and data-race defects, at compile time, on every ordinary build, and for a run-time cost the measurement bounds at a small fraction of one operation.

## What it does not buy: the part most likely to be dropped when the result is summarised

**Protection from the host**
One address space: a C++ memory error can corrupt Rust-owned data however checked the Rust side was.

**A smaller concurrency review**
Lock-ordering discipline and review for composed-but-not-atomic operations stay just as necessary.

**A free first component**
The dominant cost is human. Treat the first as a training investment and instrument it.

Sequence the migration so that components sharing the most mutable state cross over together rather than one at a time.